

## Technical Section

TARig: Adaptive template-aware neural rigging for humanoid characters<sup>☆</sup>Jing Ma, Dongliang Zhang<sup>\*</sup>

College of Computer Science and Technology, Zhejiang University, China

## ARTICLE INFO

## Article history:

Received 13 January 2023  
 Received in revised form 13 March 2023  
 Accepted 29 May 2023  
 Available online 9 June 2023

## Keywords:

Automatic rigging  
 Deep-learning  
 Template skeleton  
 Humanoid character

## ABSTRACT

We present *TARig*, a template-aware neural rigging method, to automatically generates industry-standard and high-quality skeleton and skin weights for humanoid characters. The resulting skeleton consists of a widely-used humanoid template with 21 template joints and associate secondary joint sets which are generated adaptively for handling auxiliary components. To explicitly utilize the anatomic consistency among humanoid characters, we incorporate the manually annotated anatomic label of each template joint to fully supervise the rigging process in an end-to-end training scheme. Together with a shared graph neural network backbone, such semantic constraint ensures to generate precise joint positions and accurate skin weights simultaneously. Beyond the pre-defined topology of the template skeleton, we additionally learn a boneflow field to further determine the inner connections for each secondary joint set and avoid counter-anatomical skeleton construction. Extensive experiments demonstrate *TARig* generates high-quality rigging results and outperforms other state-of-the-art auto-rigging methods in terms of skeleton generation and skin weight estimation for humanoid characters with 8~20 times speedup.

© 2023 Elsevier Ltd. All rights reserved.

## 1. Introduction

Animation-ready characters with high-quality rigs become increasingly important to video games, films, AR/VR applications, etc. Skeleton topology, joint positions, and skin weights, are three important factors for driving diverse and vivid animation sequences. The skeleton topology, including reasonable number of joints and their connections, usually depends on the preferred animation intent and dominates the level of detail for character animations. Carefully tuning joint positions and corresponding skin weights help on generating authentic animations and avoid deformation artifacts. Although manually creating such industry-standard and high-quality rigs takes great efforts, it becomes a standard workflow for articulated character animation over the past few decades. However, it takes years for an animator to master such skill and gather enough experience. As a result, automatically rigging a 3D character is a crucial technique for constantly enhancing the production efficiency in industry.

Pioneering automatic rigging works [1–3], such as Pinocchio [3], use different skeletonization algorithms [4,5] to extract the medial axis graph or medial centers of a input mesh to determine possible joint locations and then align a skeleton template to the joint hypothesis. Because these geometry-based methods lack of

anatomic priors, the predicted joints are not guaranteed to be anatomically compatible with the target character, especially for which the character's body proportion is largely different from the skeleton template (see Fig. 1(b)). Recently, many automatic rigging methods have been proposed for handling arbitrary 3D characters [6–8]. RigNet [7] identifies skeleton joints through predicting the displacements of vertices towards neighboring joints via graph neural networks. While training in a large and diverse collection of rigged models, it is able to capture animators' intuition about underlying animation intent and are not restricted by shape categorization or fixed skeleton structure [7]. However, the skeleton topology is unconstrained, as shown in Fig. 1(c), it takes extra manpower to modify the predicted skeleton when animating a target character in practice.

Combining the benefit of geometry-based methods and data-driven methods, we propose a novel neural rigging method, *TARig*, to handle the most important category, humanoid character, with an adaptive template-aware skeleton representation. Our skeleton consists of a widely-use humanoid template with 21 template joints (head, neck, shoulder, etc.) and associate secondary joint sets which are generated adaptively for representing other auxiliary components, such as clothes, hairs, tails, etc, as shown in Fig. 3. We define a boneflow field which identifies the direction from child joints to their parent to further determine the inner topology for each secondary joint set, as shown in Fig. 4. With such template adaptive representation, existing animation sequences created on the standard humanoid template can be

<sup>☆</sup> This article was recommended for publication by Dr. Y. Dobashi.

<sup>\*</sup> Corresponding author.
E-mail address: [dzhang@zju.edu.cn](mailto:dzhang@zju.edu.cn) (D. Zhang).

directly applied to the target character while the secondary joints preserve the flexibility for driving other auxiliary components.

Upon the adaptive template skeleton, we incorporate three individual neural modules, namely joint module, bone module, and skinning module, to predict the attention maps of the template and secondary joints, the boneflow field, and the skinning weights, as shown in Fig. 2. We use a shared graph neural work as the backbone for the three aforementioned modules, which extracts deep vertex features from the input mesh. Following the backbone, the joint module learns the attention map consisting of the probability of a vertex being influenced by its corresponding joint, and then regresses joint positions from the attention vertices. The bone module learns the boneflow field, and also establishes the skeleton using the predicted joint positions, fixed template topology, and the boneflows. Meanwhile, the skinning module learns the skinning weight from the deep vertex features and binds the vertices to the skeleton.

Thanks to our template definition, we can regress the joint position directly from the corresponding attention map and run the three modules parallelly, which greatly reduces the running time. Moreover, we manually annotate 21 joint categories for all characters in our dataset, which helps the neural network learn body anatomy better during the training time and improve the prediction accuracy during the inference time. Extensive experiments demonstrate the benefits of our approach. We achieve better accuracy in less running time than the competing methods [3,7,9,10]. Quantitatively, our method outperforms counterpart methods in all metrics of skeleton and skinning prediction, and it is significantly faster than the state-of-art method [7]. Qualitatively, the skeleton structure is more reasonable and the skinning weight yields better deformation quality. In summary, our contributions include

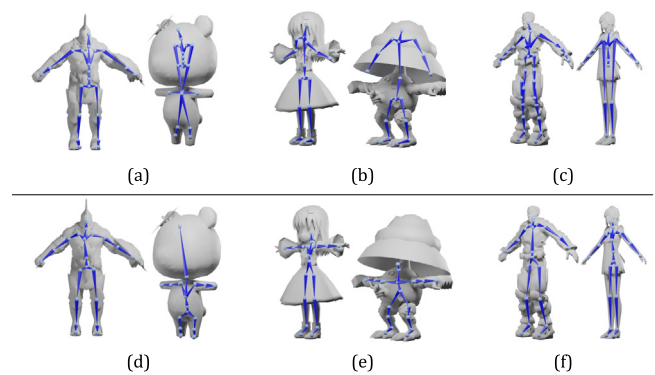
- The first adaptive template-aware neural-rigging architecture to generate the industry-standard rigged character for a humanoid mesh.
- The novel proposal of adaptive template skeleton, which improve the overall prediction accuracy as well as shorten the running time significantly.
- The introduction of the boneflow field enhances the validity of the skeleton structure, so that the constructed skeleton is ready for animation with fewer manual editions.

## 2. Related work

### 2.1. Skeleton construction

In the stage of skeleton construction, animators usually embed a template skeleton into the mesh according to the body anatomy and add auxiliary joints according to the expected deformation. Maya Quick Rig Tool [11] automates the embedding process, but the algorithm produces the wrong embedding for characters that are asymmetrical or not shaped in human body proportion, as shown in Fig. 1a. Mixamo Auto-Rigger [12] adopts a semi-auto embedding approach, despite being accurate, animators need to manually specify the key joint locations one by one. Pinocchio [3], the pioneering work in automatic rigging, extracts the medial axis graph from the input mesh, and embeds the template skeleton to the graph by optimizing a penalty function. However, the embedding procedure tends to fail when the input shape is not structurally similar to the template skeleton, as shown in Fig. 1b.

Recent data-driven approaches [6–8] demonstrate superiority in skeleton generation task. The representative work, RigNet [7], follows the Pinocchio spirit of medial axis extraction and improves the joint placement through attention prediction. However, there are three artifacts of RigNet: (1) the joint attention is not anatomy-aware, leading to misplaced, missing, or redundant



**Fig. 1.** Comparison on skeleton construction results. (a) Maya Quick Rig Tool. (b) Pinocchio [3]. (c) RigNet [7]. (d-f) Ours.

joints shown in Fig. 1c, furthermore, manual annotations are required for motion retargeting task since the predicted joints of RigNet lack semantic labels, (2) there exist many uncertainties in the bone connection network, leading to invalid topology that requires extra manpower to turn the skeleton to be valid, and (3) the iterative contraction-and-clustering method for skeleton generation is time-consuming, and the running time exceeds 20 min for a typical game character with 12000 vertices, which is unacceptable in the interactive scenario of animation production. Our template-aware boneflow-guided method discards the contraction-and-clustering procedure, leading to topology-valid skeleton (shown in Fig. 1d,e,f) with less time cost.

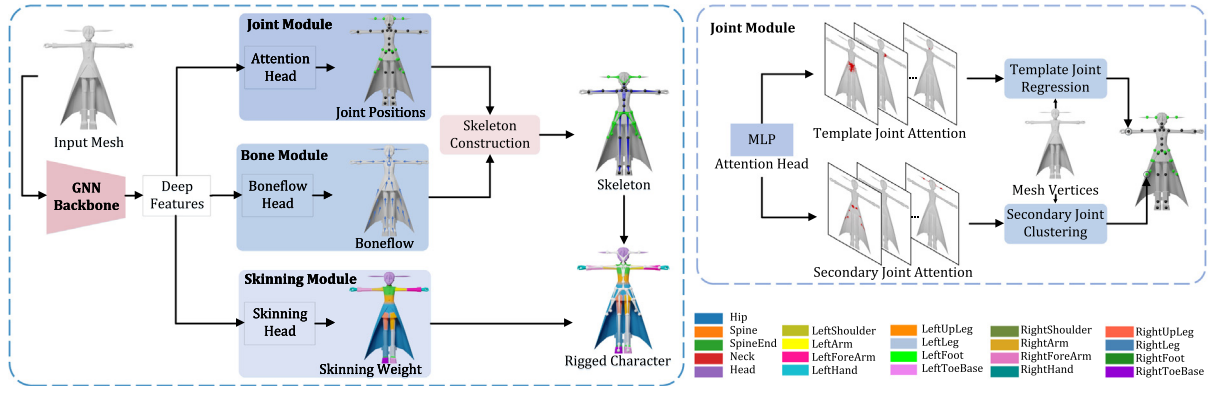
### 2.2. Skinning

In the stage of skinning, animators paint skin weights based on anatomic understandings and deformation expectations. Given the skinning weight and joint transformations, the deformed shape is usually calculated by Linear Blend Skinning (LBS) [13] or Dual Quaternion Skinning (DQS) [14]. Painting skin weights requires animators to have keen insight into body parts and involves repetitive trying and modifying. Many geometric methods [3,9,15–19] have been proposed to automate the skinning process. However, it is difficult for these geometric methods to distinguish close vertices that are semantically different because these approaches are all purely based on vertex-to-bone distance without incorporating human's perception. Moreover, apart from GVB [9] and SparseGVB [17], most works [3,15,16,18,19] are limited to the single-component manifold mesh.

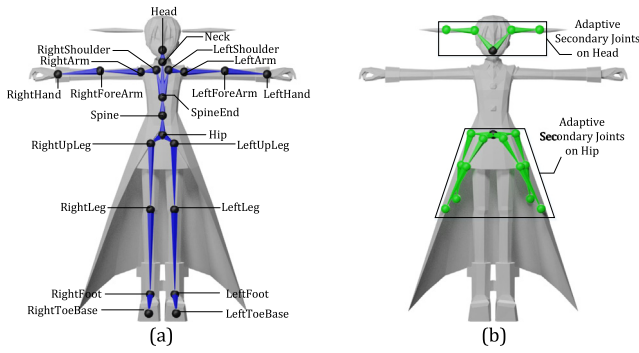
Recent data-driven methods [7,10,20] can incorporate animators' skinning experience from a wide range of 3D characters and apply to non-manifold meshes. However, these methods inherit the vertex-to-bone idea that the skinning weight is predicted according to the distance between vertices and bones, and the artifact that close parts affect each other is not well solved. In contrast, our method learns body semantics across the data through annotated joint labels, which alleviates the influence issue of near vertices and demonstrates better deformation quality.

### 2.3. Graph neural networks

Graph Neural Network (GNN) has been widely used in data processing tasks [21–24]. It is gaining popularity in geometric data – point clouds and meshes on both spectral [25–27] and spatial domains [7,10,28–31]. We adopt EdgeConv [31] operator to extract the vertex features from input meshes. The learned deep features are used to predict the joint attention, boneflow, and the skinning weight in a single forward pass.



**Fig. 2.** Method overview. Given a character mesh, a GNN backbone extracts deep vertex features, which are sent to three heads for the prediction of joint attention, boneflow, and skinning weights. The joint positions are regressed for template joints (black points) and clustered for secondary joints (green points) from attention vertices. The skeleton is built from the joints and boneflow. The rigged model is produced after combining the skinning weight.



**Fig. 3.** Adaptive template skeleton. (a) We define 21 template joints widely used in the industry for humanoid rigging. (b) Adaptive secondary joints associated with the template Head joint and the template Hip joint.

### 3. Overview

Given a humanoid mesh, our goal is to predict a rigged character conforming to the industry standard with adaptive control so that it will be easy for animators to produce high-quality animation. Motivated by the goal of usability, flexibility, and accuracy, we propose a template-aware architecture highlighted by an adaptive template skeleton and a boneflow field. Our method assumes the input character is upright and front-facing.

In this paper, we use the following notations:  $\mathbf{V}$  and  $\mathbf{X}$  denote the vertex positions and vertex features respectively,  $\mathbf{H}$  and  $\mathbf{J}$  denote joint attention and joint positions respectively,  $\mathbf{B}$  denotes the boneflow among joints,  $\mathbf{S}$  denotes the skinning weight,  $\mathbf{W}$  denotes the learned parameters of the neural network,  $n$  denotes the number of mesh vertices,  $m$  denotes the number of template joints, and  $\parallel$  denotes the concatenation operator.

### 4. Method

In this section, we present details of our architecture shown in Fig. 2: a shared backbone extracting the deep vertex features, the joint module for joint attention and joint positions, the bone module for boneflow prediction and skeleton construction, and the skinning module for weights prediction and skin binding.

#### 4.1. Backbone network

Given an input mesh, we start with learning vertex features with a shared GNN backbone. The input to the backbone are

vertex positions, vertex normals, and edges. The output of the backbone are learned vertex features  $\mathbf{X} \in \mathbb{R}^{n \times k}$  with  $k$  channels.

The basic operator of our backbone network is *EdgeConv* operator, which proves to be effective in segmentation and classification task [31]. An *EdgeConv* operator for vertex  $\mathbf{v}_i$  on layer  $l$  is represented by

$$\begin{aligned} \mathbf{x}_i^{l+1} &= \text{EdgeConv}(\mathbf{x}_i^l, \mathcal{N}(i); \mathbf{W}^l) \\ &= \max_{j \in \mathcal{N}(i)} \text{MLP}(\mathbf{x}_i^l, \mathbf{x}_j^l - \mathbf{x}_i^l; \mathbf{W}^l) \end{aligned} \quad (1)$$

where  $\mathcal{N}(i)$  denotes neighbors of vertex  $\mathbf{v}_i$ , *MLP* means Multi-Layer-Perceptron,  $l$  denotes the layer number, and  $\mathbf{W}^l$  denotes the learned parameters of *MLP* at layer  $l$ . In the following, we omit the suffix  $l$  and  $l + 1$  for simplicity. We adopt *GMEdgeConv* operator [7] that aggregates edge features from both topological neighbors and geodesic neighbors

$$\mathbf{x}_i = \text{MLP}(\text{EdgeConv}(\mathbf{x}_i, \mathcal{N}_t(i)) \parallel \text{EdgeConv}(\mathbf{x}_i, \mathcal{N}_g(i))) \quad (2)$$

where  $\mathcal{N}_t(i)$  denotes the topological 1-ring neighbors from mesh edges, and  $\mathcal{N}_g(i)$  denotes the geodesic neighbors inside a sphere around a vertex, which we use the sphere with 0.06 radius. By including both topological neighbors and geodesic neighbors into the receptive field of convolution, the network is proved to learn better representations for the non-manifold mesh. The GNN backbone is similar to the one proposed by [7], please refer Appendix Table 7 for the detail of the architecture.

#### 4.2. Joint module

Our joint module predicts template joints and secondary joints. The prediction of template joints enables basic rigging on indispensable joints such as the shoulder, elbow, knee, etc. The prediction of secondary joints enables adaptive rigging on auxiliary parts such as tails, hairs, clothes, etc. In the following, we present details of the joint module.

**Attention Map Construction.** For an input character, we predict the template attention  $\mathbf{H}_t \in \mathbb{R}^{n \times m}$  and the secondary attention  $\mathbf{H}_s \in \mathbb{R}^{n \times m}$  with  $\mathbf{H}_{i,j} = 1$  indicating that the  $i$ th vertex contributes to the prediction of the  $j$ th joint, as shown in red regions of Fig. 2. The attention map is created by performing ray-mesh intersections on each joint, we only keep hit vertices that are inside the joint's influential radius, as shown in Fig. 11.

**Attention Map and Joints Prediction.** Given the mesh features  $\mathbf{X}$  from the backbone network, we employ a 3-layer MLP for prediction of joint attention

$$\hat{\mathbf{H}} = \text{sigmoid}(\text{MLP}(\mathbf{X}; \mathbf{W}_h)) \quad (3)$$

where  $\hat{\mathbf{H}} \in \mathbb{R}^{n \times 2m}$  is the concatenation of the template attention  $\hat{\mathbf{H}}_t$  and the secondary attention  $\hat{\mathbf{H}}_s$ , and  $\mathbf{W}_h$  denotes the learned parameters of the attention head. By encoding the anatomic-consistency among humanoid characters into  $\mathbf{W}_h$ , our method enables more precise joint placement. The  $j$ th template joint is regressed from attention  $\hat{\mathbf{H}}_{i,j}$  and vertex positions  $\mathbf{V}$

$$\mathbf{J}_j = \frac{\sum_{i=1}^n \hat{\mathbf{H}}_{i,j} \mathbf{V}_i}{\sum_{i=1}^n \hat{\mathbf{H}}_{i,j}} \quad (4)$$

where  $\hat{\mathbf{H}}_{i,j} / \sum_{i=1}^n \hat{\mathbf{H}}_{i,j}$  is the normalized attention value of  $i$ th vertex to  $j$ th joint. Since we directly regress the template joints from attention vertices in a single forward pass, our method achieves several times speedup.

The secondary attention from the last  $m$  channels are used for adaptive secondary joints extraction. Since the amount of secondary joints is uncertain, we only supervise secondary attention maps without regressing joint positions during the training time. During the inference time, we filter important vertices by thresholding the secondary attention map, and use the filtered vertices as the input to meanshift clustering, refer Eq. (6) of [7]. The cluster centers are then set as the positions of secondary joints.

**Attention Loss and Joint Loss.** We supervise the learning of attention maps and template joints with ground truth values  $\mathbf{H}$  and  $\mathbf{J}$ . For attention maps, we employ dice loss due to its superiority in segmenting the small region of interest [32]

$$\mathcal{L}_{\text{attention}} = 1 - \frac{2 \sum_{i,j} (\hat{\mathbf{H}}_{i,j} \cdot \mathbf{H}_{i,j})}{\sum_{i,j} \hat{\mathbf{H}}_{i,j}^2 + \sum_{i,j} \mathbf{H}_{i,j}^2} \quad (5)$$

where the ground truth attention  $\mathbf{H}_{i,j} \in \{0, 1\}$  is a discrete integer of either 0 or 1, and the predicted attention  $\hat{\mathbf{H}}_{i,j} \in [0, 1]$  is a continuous real value representing the importance of  $i$ th vertex to  $j$ th joint.

For template joint positions, we use Mean Squared Error (MSE) Loss and Chamfer Distance (CD) Loss

$$\mathcal{L}_{\text{joints}} = \mathcal{L}_{\text{mse}} + \mathcal{L}_{\text{cd}} \quad (6)$$

$$\mathcal{L}_{\text{mse}} = \frac{1}{|\mathbf{J}|} \sum_j \|\mathbf{J}_j - \hat{\mathbf{J}}_j\| \quad (7)$$

$$\mathcal{L}_{\text{cd}} = \frac{1}{|\mathbf{J}|} \sum_j \min_k \|\mathbf{J}_j - \hat{\mathbf{J}}_k\| + \frac{1}{|\hat{\mathbf{J}}|} \sum_k \min_j \|\hat{\mathbf{J}}_k - \mathbf{J}_j\| \quad (8)$$

where  $\mathbf{J}$  denotes the annotated ground-truth joint positions,  $\hat{\mathbf{J}}$  denote the predicted joint positions, and the number of annotated joints  $|\mathbf{J}|$  is usually less than or equal to the number of predicted joint positions  $|\hat{\mathbf{J}}|$ .

#### 4.3. Bone module

We define the boneflow field  $\mathbf{B} \in \mathbb{R}^{n \times 3}$  shown in Fig. 4 to guide the skeleton construction, which are normalized direction vectors pointing from the child joint to its parent joint.

**Boneflow Prediction.** We predict boneflows through a 3-layer MLP head. The head takes backbone features  $\mathbf{X}$  as input and outputs 3 channels of direction fields in the last layer

$$\hat{\mathbf{B}} = \text{MLP}(\mathbf{X}; \mathbf{W}_b) \quad (9)$$

where  $\mathbf{W}_b$  denotes the learned parameters of the boneflow head and  $\hat{\mathbf{B}}$  denotes the predicted boneflows that are input to the skeleton construction algorithm.

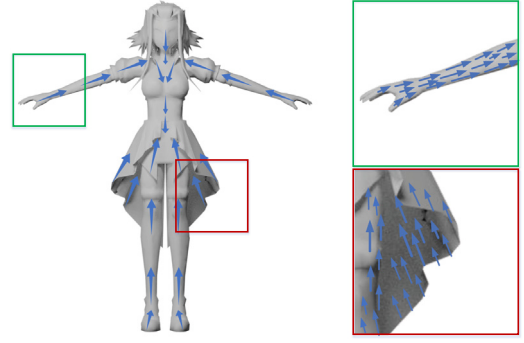


Fig. 4. Boneflow field. We define boneflows pointing from the child joint to its parent joint to guide the skeleton construction.

**Boneflow Loss.** The boneflow loss is the MSE between the ground truth boneflows  $\mathbf{B}$  and the predicted boneflows  $\hat{\mathbf{B}}$

$$\mathcal{L}_{\text{boneflow}} = \frac{1}{n} \sum_{i=1}^n \|\mathbf{B}_i - \hat{\mathbf{B}}_i\| \quad (10)$$

where  $n$  is the number of mesh vertices, and  $\mathbf{B}_i$  is the boneflow defined on  $i$ th vertex.

**Skeleton Construction.** The template joints are connected using the fixed topology of the template skeleton. The bone relationships among secondary joints are formed using the Euclidean distance and the boneflow direction

$$\begin{aligned} \text{cost}_{i,j} &= \text{cost}_{\text{euc}}(i, j) + \text{cost}_{\text{boneflow}}(i, j) \\ &= \|\mathbf{J}_j - \mathbf{J}_i\| + \frac{1}{|\mathbb{C}_i|} \sum_{i \in \mathbb{C}_i} \arccos \hat{\mathbf{B}}_i \cdot \hat{\mathbf{J}}_{ij} \end{aligned} \quad (11)$$

where  $\mathbb{C}_i$  is the collection of vertices in the attention region of the  $i$ th joint as shown in Fig. 11.  $\text{cost}_{i,j}$  represents the total cost of traveling from  $i$ th joint to  $j$ th joint, the smaller the value is, the more likely to form a bone between them.  $\text{cost}_{\text{boneflow}}(i, j)$  measures the angle deviation between the predicted boneflows  $\hat{\mathbf{B}}_i$  and the unit directions  $\hat{\mathbf{J}}_{ij} = (\mathbf{J}_j - \mathbf{J}_i) / \|\mathbf{J}_j - \mathbf{J}_i\|$  from  $i$ th joint to  $j$ th joint. To form the bones among secondary joints and their corresponding template joint, we first attach all secondary joints to the template joint, then re-attach the secondary joint to another secondary joint if the cost between them is smaller than the current one. With the known template topology and boneflow-guided connections from secondary joints to the template joint, the generated skeleton is structurally valid and requires fewer post-processings for animation usage.

#### 4.4. Skinning module

We predict the skinning weight matrix defined on template joints. Since some 3D characters in our dataset may not have  $m$  template joints, we perform hierarchical weight merging to form the ground truth weight matrix. Specifically, given a raw skinning weight matrix  $\hat{\mathbf{S}} \in \mathbb{R}^{n \times p}$  defined on the original  $p$  joints, the ground truth skinning weight matrix  $\mathbf{S} \in \mathbb{R}^{n \times m}$  is calculated by merging the influenced vertices of the secondary child joint to the template parent joint.

**Skinning Prediction.** The skinning head is a 3-layer MLP that takes backbone features  $\mathbf{X}$  as input and outputs  $m$  channels for skinning weights on template joints.

$$\hat{\mathbf{S}} = \text{MLP}(\mathbf{X}; \mathbf{W}_s) \quad (12)$$

where  $\hat{\mathbf{S}}$  denotes the predicted skinning weight and  $\mathbf{W}_s$  denotes the learned parameters of the skinning head. Since each anatomic

**Table 1**  
Statistics of *ModelResource-Ann* dataset.

| Vertex number |      |      | Edge length |        |        | Joint number |     |     |
|---------------|------|------|-------------|--------|--------|--------------|-----|-----|
| min           | max  | avg  | min         | max    | avg    | min          | max | avg |
| 1000          | 4941 | 2057 | 0.0095      | 0.0995 | 0.0354 | 6            | 48  | 25  |

part corresponds to a dedicated column of  $\hat{\mathbf{S}}$ , our method encourages mutual learning from different characters and the predicted skinning weight results in better deformation quality.

**Skinning Loss.** Skinning on template joints is similar to mesh segmentation of different body parts since most vertices in a rigid part correspond exactly to one bone. Thus, we use cross-entropy loss to supervise the training

$$\mathcal{L}_{\text{skinning}} = \frac{1}{n} \sum_i \sum_j \mathbf{s}_{i,j} \log \hat{\mathbf{s}}_{i,j} \quad (13)$$

where  $\hat{\mathbf{s}}_{i,j}$  denotes the predicted influence of  $j$ th joint to  $i$ th vertex, and  $n$  is the number of mesh vertices.

**Skinning Computation** We compute the skinning weight for the whole skeleton given the predicted template skinning weight. Specifically, we find the template joint having secondary joints as descendants, and re-bind its influential vertices to the joints set formed by the template joint and its descendants. We use the closest-hierarchy skinning for the re-binding, more advanced skinning algorithms [7,9] can also be used for better deformation quality. Finally, we impose the sparsity constraint and affinity constraint [15] during the post-filtering step.

#### 4.5. Total loss

We learn joint attention, joint positions, boneflows, and skinning weights jointly from the mesh features extracted by the shared GNN backbone. The total loss is defined as a weighted summation of the four losses

$$\mathcal{L} = \lambda_a \mathcal{L}_{\text{attention}} + \lambda_j \mathcal{L}_{\text{joints}} + \lambda_b \mathcal{L}_{\text{boneflow}} + \lambda_s \mathcal{L}_{\text{skinning}} \quad (14)$$

where  $\lambda$  denotes the weights to balance different losses. We find  $\lambda_a = 1$ ,  $\lambda_j = 100$ ,  $\lambda_b = 1$ , and  $\lambda_s = 1$  yield the best result through experiment.

## 5. Experiments

**Dataset.** We annotate 2046 humanoid models from *ModelResource* [7] dataset, which we call *ModelResource-Ann*. The annotation contains 21 joints that are commonly used in the animation pipeline, as shown in Fig. 3. The statistics of *ModelResource-Ann* are reported in Table 1. We split the dataset into train, test, and validation with an 80%-10%-10% proportion, yielding 1636 for train, 205 for test, and 205 for validation.

**Implementation Details.** Our framework is implemented with PyTorch [33] and Pytorch-Geometric [34] on NVIDIA GeForce RTX 3060 GPU (12 GB) and Intel Core i7-10700/2.90 GHz CPU (32 GB RAM). We use Adam optimizer and MultiStepScheduler to train the network for 100 epochs and reduce the learning rate to half after 50 epochs. We set the initial learning rate to  $5e-4$  and the batch size to 2 during training.

#### 5.1. Evaluation metrics

**Skeleton Prediction.** We follow RigNet [7] to evaluate the predicted skeleton with *CD-J2J*, *CD-J2B*, *CD-B2B*, *IoU*, *Precision & Recall*. We also evaluate the topology validity of the skeleton in terms of *GraphL1*.

(a) *CD-J2J*, *CD-J2B*, and *CD-B2B*. *CD-J2J*, *CD-J2B*, and *CD-B2B* measure the closeness between the predicted skeleton and ground-truth skeleton. Chamfer Distance (*CD*) is a common evaluation metric for two point sets, which calculates the nearest distance for each point from one point set to the other and takes the average of all closest distances. We use symmetric *CD* by default which is the mean of two bi-directional *CD* of the two point sets. *CD-J2J* measures the closeness between the ground truth joints and predicted joints. *CD-J2B* measures the closeness between predicted joints and ground truth bones. *CD-B2B* measures the closeness between predicted bones and ground truth bones.

(b) *IoU*. *IoU* (Intersection over Union) measures the similarity between the predicted skeleton and the ground-truth skeleton. We calculate the number of matched joints between the predicted and the ground-truth with a distance smaller than a certain threshold. The threshold is the influential radius of the ground truth joint, which is precomputed by intersecting the mesh with rays emitted from the joint. The *IoU* is the overlapping number divided by the total number of all joints consisting of predicted joints and ground-truth joints.

(c) *Precision & Recall*. *Precision* is the number of matched joints divided by predicted joints. *Recall* is the number of matched joints divided by the ground-truth joints.

(d) *GraphL1*. A skeleton is a directed graph that can be represented by an adjacency matrix  $\mathbf{G}$

$$\mathbf{G}_{i,j} = \begin{cases} 1, & \text{if joint } i \text{ connects to joint } j \\ 0, & \text{else} \end{cases} \quad (15)$$

We define *GraphL1* as  $|\hat{\mathbf{G}} - \mathbf{G}|/n$  with  $n$  denoting the row number of  $\mathbf{G}$ . *GraphL1* is smaller when the predicted skeleton  $\hat{\mathbf{G}}$  is more similar to the ground truth skeleton  $\mathbf{G}$ .

**Skinning Prediction.** We evaluate the accuracy of skinning prediction in terms of *Dist* [7] and *L1*.

(1) *Dist*. *Dist* (Deformation Distance) measures the Euclidean distance between the deformed mesh using the predicted skinning weight and the deformed mesh using the ground truth skinning weight under the same pose. A smaller deviation indicates better performance.

(2) *L1*. *L1* (L1 Norm of Skinning Weight) measures the difference between the predicted skinning weight and the ground truth skinning weight. A smaller value means higher accuracy.

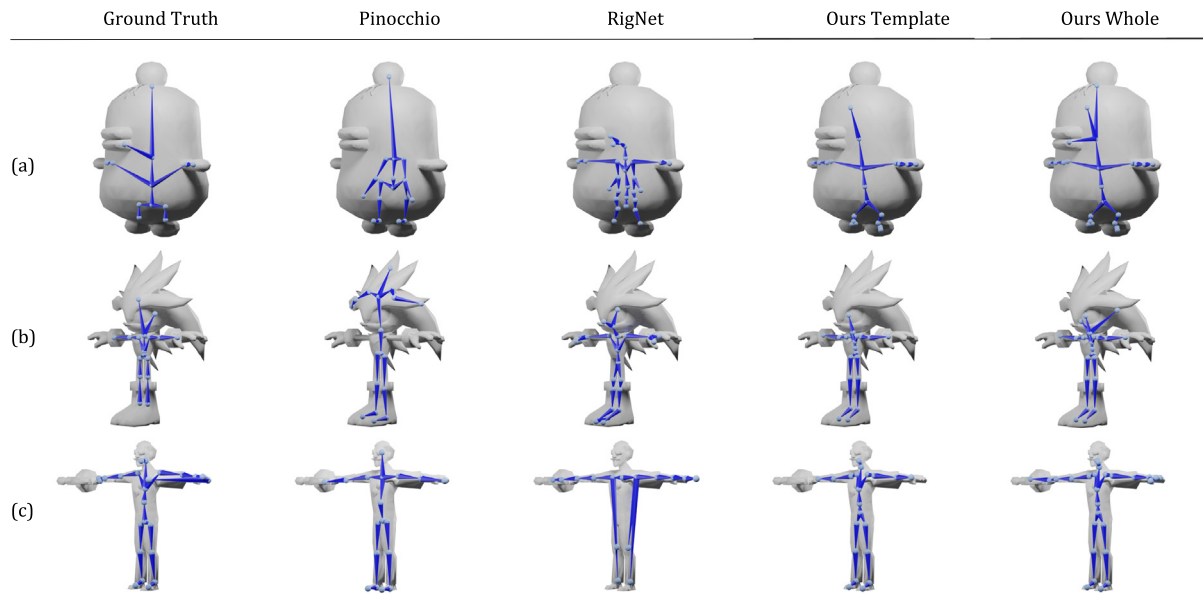
#### 5.2. Comparisons

**Skeleton Prediction.** For skeleton prediction, we compare with Pinocchio [3] and RigNet [7]. Pinocchio [3] embeds a predefined template skeleton to the watertight mesh. The author provides four templates, humanoid, short quadruped, long quadruped, and centaur. We use the author's code,<sup>1</sup> convert the non-manifold mesh into watertight, and perform the comparison using the humanoid template on the same test dataset as ours. RigNet [7] trains a large amount of data and learns the offset and attention of mesh vertices as well as the bone connection probability. The joints are extracted by attention-driven clustering over the contracted mesh, and the skeleton is constructed using bone connection probability by Minimum Spanning Tree algorithm. We use the author's code<sup>2</sup> and train/test RigNet on the same dataset split as ours.

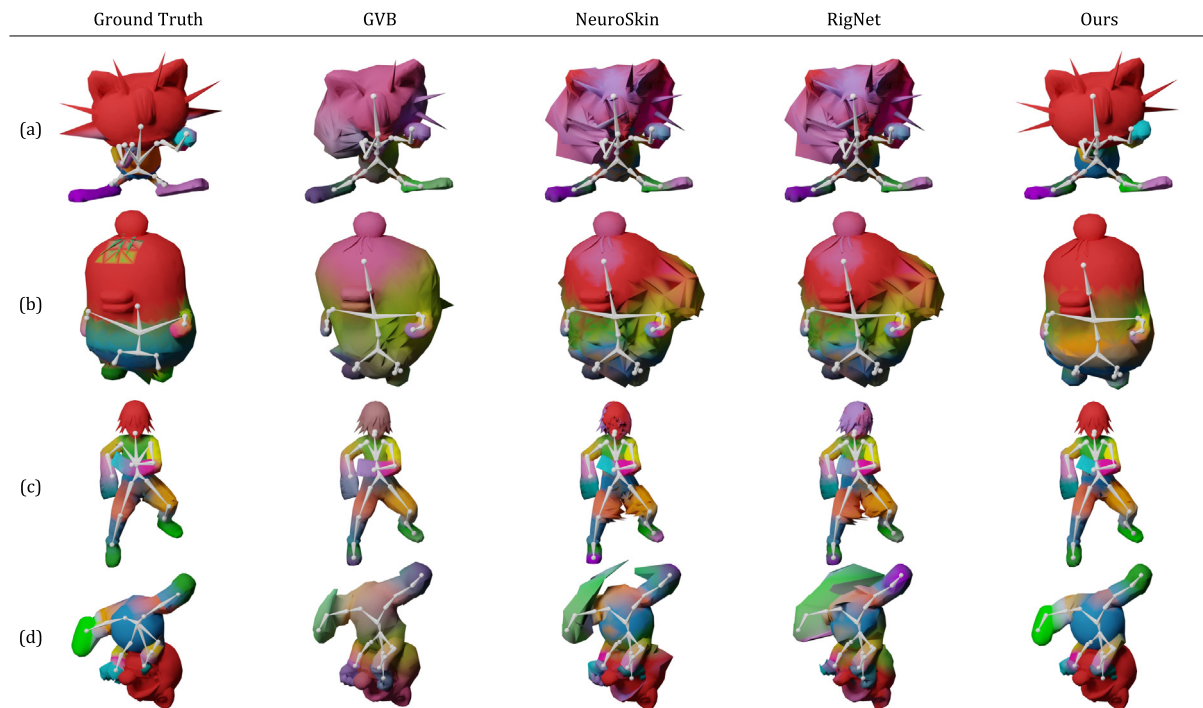
Fig. 5 shows the comparison result with the counterparts. Our method produces more plausible skeletons than the two competing methods. Pinocchio [3] sometimes produces wrong

<sup>1</sup> <https://github.com/pmolodo/Pinocchio>

<sup>2</sup> <https://zhan-xu.github.io/rig-net/>



**Fig. 5.** Skeleton comparisons with previous methods. Pinocchio produces wrong embedding for character(a)(b). RigNet generates redundant (a), misplaced joints (b) and wrong bone connections (c). The last two columns show our template skeleton and the whole skeleton after attaching the secondary joints to the template joint. It can be seen the skeleton of our method agrees more with the ground-truth.



**Fig. 6.** Skinning comparisons with previous methods. The competing methods bind the vertices to the wrong bone when two body parts are close to each other, see shoulder and head of character(a)(b), left leg and right leg of character(c)(d). The last column shows our result. It can be seen the skinning weight generated by our method leads to better deformation quality.

embedding as shown in Fig. 5(a)(b). For character(a), the arms and legs are too small to contribute the medial axis graph for skeleton construction, refer Fig. 2,3,4 of Pinocchio [3]. For character(b), the arms are too thin so that arm joints are embedded wrongly to the thick hair. It is revealed that the traditional geometric method [3] fails to capture riggers' insight for body anatomy. RigNet [7] is demonstrated to produce anatomic-aware skeleton embedding by learning from a large database. However, as shown in Fig. 5, the method produces redundant joints (a), misplaced joint positions (b), or wrong bone connections (c), which requires

additional effort for animators to turn them into valid ones. The reason causing redundancy and misplacement is because that RigNet does not impose the template constraint instead of producing uncertain amount of joints through shift-and-clustering. The cause for wrong bone connections is because that the connection probability predicted by the bone network [7] is unstable, our method introduce more robustness through template structure and vertices-based boneflows. Table 2 reports the quantitative result for template skeleton. It can be seen that our method outperforms other methods in all evaluation metrics.

**Table 2**  
Comparisons with other *skeleton prediction* methods.

| Methods   | IoU    | Prec   | Rec    | CD-J2J | CD-J2B | CD-B2B |
|-----------|--------|--------|--------|--------|--------|--------|
| Pinocchio | 39.23% | 45.58% | 37.98% | 6.37%  | 5.43%  | 3.98%  |
| RigNet    | 50.96% | 55.78% | 49.39% | 4.75%  | 2.88%  | 2.76%  |
| Ours      | 55.18% | 61.21% | 53.06% | 4.12%  | 2.33%  | 2.27%  |

**Table 3**  
Comparisons with other *skinning prediction* methods.

| Method    | Avg Dist | Avg L1 |
|-----------|----------|--------|
| GVB       | 0.0212   | 0.73   |
| NeuroSkin | 0.0366   | 0.62   |
| RigNet    | 0.0341   | 0.59   |
| Ours      | 0.0189   | 0.52   |

**Skinning Prediction.** For skinning weights prediction, we compare with GVB [9], NeuroSkin [10], and RigNet [7]. We adopt Maya's GVB implementation [11] and use the default parameters in our experiment. As for NeuroSkin [10], we implement the code based on RigNet's implementation by replacing the network architecture with AGNN [35] and replacing the volumetric geodesic vertex-to-bone distance with Euclidean distance. For RigNet [7], we adopts the official released code. In all comparisons, we use the same train-test split as ours and evaluate the competing methods on the L1 skinning difference (Avg L1) and L1 deformation distance (Avg Dist). During the evaluation of deformation distance, we randomly rotate the joint 100 times subject to a uniform distribution within  $[-\pi/6, \pi/6]$ . Since the characters in ModelResource-Ann dataset may not contain all 21 template joints, we only select the joints in the template joints to rotate. Notice that we use the template skeleton predicted by our *joint module* as input to GVB, NeuroSkin, and RigNet for fair comparisons.

Fig. 6 shows the comparison result with competing methods. Our method yields better deformation quality than its counterparts. Notice all three competing methods falsely bind some head vertices to the shoulder for character(a)(b), NeuroSkin and RigNet falsely distinguish the influential vertices of left leg and right leg for character(c)(d). The artifact appears when two body parts are close to each other. Our method alleviate the artifact to certain degree, as shown in the last column of Fig. 6. We speculate this is because that our method is template-based, the annotated joints help the network learn better about the body anatomy, thus, leading to more accurate result.

Table 3 reports the quantitative result. It can be seen that our method achieves smaller deformation deviation and skinning difference than the competing methods.

#### Secondary Joints and Deformation

In the following, we present results on secondary joints and make visual comparisons with counterparts. Fig. 7 shows results for skeleton prediction. Pinocchio [3] does not generate secondary joints due to the pre-defined skeleton structure is fixed. RigNet also fails to predict the secondary joints since the method does not pay as much attention for auxiliary parts as our method but instead mixes auxiliary parts with main parts during training and inference. Fig. 8 shows results for skinning prediction. Our method performs better than other methods on the main body parts, and it achieves similar results with competing methods on the auxiliary parts. Since the number of auxiliary parts is relatively small in our dataset, the result is not as good as the main body parts.

**Running Time.** We report the total time of rigging a character including joint prediction, bone connection, and skinning in Table 8 (Appendix). Our method runs significantly faster than RigNet, as shown in Fig. 9.

**Table 4**  
Share backbone or separate backbone.

| Methods  | IoU    | Prec   | Rec    | CD-J2J | CD-J2B | CD-B2B |
|----------|--------|--------|--------|--------|--------|--------|
| Separate | 55.65% | 61.72% | 53.43% | 4.08%  | 2.26%  | 2.25%  |
| Share    | 55.18% | 61.21% | 53.06% | 4.12%  | 2.33%  | 2.27%  |

**Table 5**  
Joint module ablation.

| Strategies           | IoU    | Prec   | Rec    | CD-J2J |
|----------------------|--------|--------|--------|--------|
| <i>tpl neighbors</i> | 53.34% | 58.78% | 51.55% | 4.56%  |
| <i>geo neighbors</i> | 52.92% | 58.01% | 50.82% | 4.61%  |
| w/o MSE              | 48.28% | 53.12% | 46.89% | 4.58%  |
| w/o CD               | 54.80% | 59.98% | 52.99% | 4.45%  |
| Full                 | 55.18% | 61.21% | 53.06% | 4.12%  |

Specifically, it runs around 8-times faster than RigNet for a character with 6000 vertices. For a typical game character with 12000 vertices, our method only needs 45 s while RigNet takes 27 min, which is unacceptable in the real scenario of user interaction. The time cost of RigNet largely attributes to the iterative contraction-and-clustering method of locating joints, which increases when the mesh becomes dense. Our method directly regresses joint positions in a single forward pass, thus can save lots of time.

#### 5.3. Ablation study

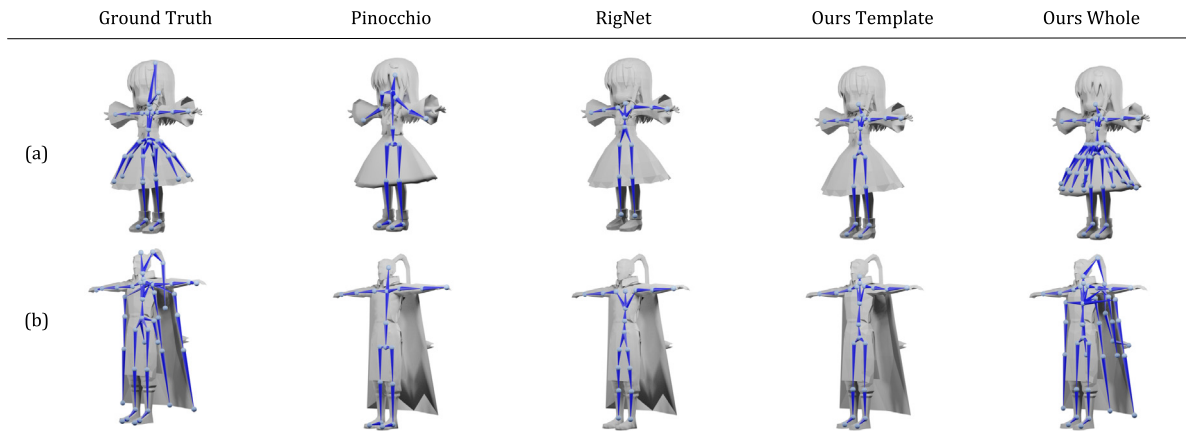
In this section, we carry out a series of experiments to explore the impact of different components in the proposed framework.

**Share Backbone.** The other alternative to our architecture is that each head (joint attention, boneflow, skinning) has its own backbone network, which has three times more parameters than the current share backbone architecture and is expected to yield better results. Table 4 presents the comparisons between the *Separate Backbone* design and the current *Share Backbone* design on the template joints prediction. It can be seen the *Separate Backbone* architecture does not improve the prediction accuracy so much. Therefore, we use the lightweight *Share Backbone* architecture in this paper.

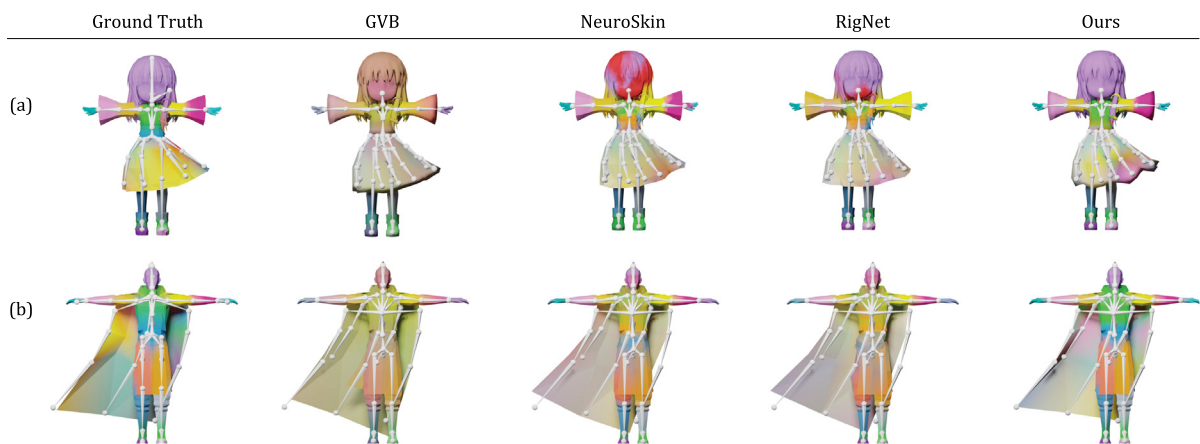
#### Joint Module.

Table 5 presents the evaluation result of different components of joint module. *tpl neighbors* refers to the topological 1-ring neighbors of vertex, which means the *EdgeConv* [31] operator only aggregates information from vertices directly connected by mesh edges. *geo neighbor* refers to the neighbors inside the geodesic sphere (within a radius threshold 0.006) of the vertex, which means the *EdgeConv* [31] operator only acts on the geodesic neighbors. It can be seen from Table 5 that aggregation only on the *tpl neighbors* is slightly better than aggregation only on the *geo neighbors*. But combining both *tpl neighbors* and *geo neighbors* gives the best result, which may be explained by the fact that we work on non-manifold meshes, and the features of *geo neighbors* complement the features of the *tpl neighbors*. w/o MSE means we omit the Mean Squared Error loss (Eq. (7)) from the joint supervision. w/o CD means we omit the Chamfer Distance loss (Eq. (8)) from the joint supervision. It can be seen that *mse loss* plays a crucial role in regressing accurate joint positions. *CD loss* is helpful to place the joints to the medial axis center of the mesh.

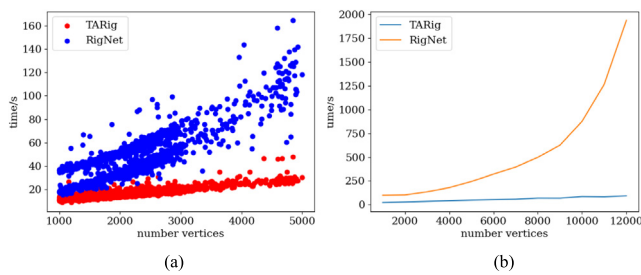
**Bone Module.** Table 6 evaluates different strategies in guiding skeleton construction: (1) *EucCost* connects joints according to the Euclidean distance between them, (2) *BoneflowCost* connects joints according to the angle deviation from the boneflow direction, and (3) *Full* connects joints by considering both *EucCost* and *BoneflowCost*, as mentioned in Section 4.3. In this experiment,



**Fig. 7.** Skeleton comparisons on secondary joints. Pinocchio and RigNet fail to predict the secondary joints, while our method is able to generate the secondary joints on auxiliary parts such as dress, hair, and cloak.



**Fig. 8.** Skinning comparisons on secondary joints. Our method produce more accurate skinning weight on the main body parts than GVB, NeuroSkin, and RigNet. It achieves similar results with competing methods on the auxiliary parts.

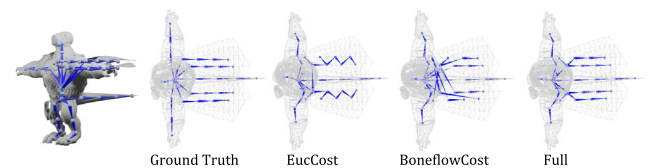


**Fig. 9.** Time Comparisons. (a) Test TARig on all characters of the dataset. (b) Test TARig 10 times on 1 Mixamo character with increasing number of vertices since the dataset only contain meshes within 1000–5000 vertices as mentioned in Table 1.

we use ground truth joint positions as input. By eliminating the wrong predicted joint positions from evaluation, we only assess the connection part of our algorithm. It can be seen that only using *EucCost* gives the worst topology while introducing the *boneflow* improves the result, as shown in Table 6 and Fig. 10.

## 6. Conclusion and future work

We present a learning-based method that predicts a standard rigged character for the non-manifold mesh in seconds, which



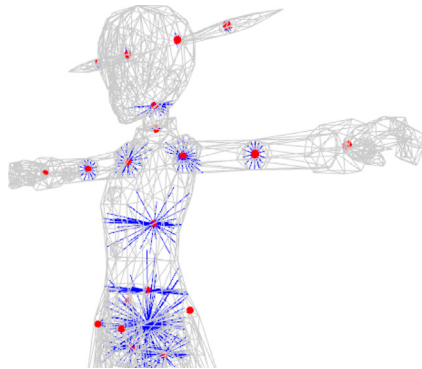
**Fig. 10.** Bone module ablation. Different strategies for bone connection. *EucCost*: create bones according to the Euclidean distance between joints. *BoneflowCost*: create bones according to the boneflow directions between joints. *Full*: create bones considering both the *EucCost* and the *BoneflowCost*.

| Table 6<br>Different strategies for bone connection (whole skeleton → secondary skeleton). |        |         |
|--------------------------------------------------------------------------------------------|--------|---------|
| Strategies                                                                                 | CD-B2B | GraphL1 |
| EucCost                                                                                    | 0.903% | 0.485   |
| BoneflowCost                                                                               | 0.605% | 0.414   |
| Full                                                                                       | 0.724% | 0.409   |

requires less manpower to produce high-quality animations. Our contributions include: (1) we present the first adaptive template-aware neural-rigging architecture for humanoid character and the rigged character can be directly used for animation with fewer post-modifications, (2) we propose an adaptive template skeleton, which improves the overall rigging accuracy in terms of

**Table 7**  
Network architecture.

| Name            | Layers                                                              | I/O Channels | I/O Names                       |
|-----------------|---------------------------------------------------------------------|--------------|---------------------------------|
| GNN Backbone    | GMEdgeConv                                                          | 6/64         | $\mathbf{X}_{in}/\mathbf{X}_1$  |
|                 | GMEdgeConv                                                          | 64/256       | $\mathbf{X}_1/\mathbf{X}_2$     |
|                 | GMEdgeConv                                                          | 256/512      | $\mathbf{X}_2/\mathbf{X}_3$     |
|                 | $\mathbf{X}_1 \parallel \mathbf{X}_2 \parallel \mathbf{X}_3$        | –/832        | –/ $\mathbf{X}_4$               |
|                 | MLP+MaxPool                                                         | 832/1024     | $\mathbf{X}_4/\mathbf{X}_{gib}$ |
|                 | $\mathbf{X}_{in} \parallel \mathbf{X}_4 \parallel \mathbf{X}_{gib}$ | –/1859       | –/ $\mathbf{X}$                 |
| Joint Attention | MLP                                                                 | 1859/42      | $\mathbf{X}/\mathbf{H}$         |
| Bone Flow       | MLP                                                                 | 1859/3       | $\mathbf{X}/\mathbf{B}$         |
| Skinning        | MLP                                                                 | 1859/21      | $\mathbf{X}/\mathbf{S}$         |

**Fig. 11.** Joint Attention Map Construction.

joints locations and skin binding, and (3) we innovatively introduce the boneflow field to guide the bone connections in adaptive parts, generating more plausible skeleton structure together with the template part.

Our method does have limitations. (1) As with other data-driven methods, the accuracy of our method is restricted by the dataset used to train the network. The skeleton in the Model-Resource-Ann dataset is sometimes not standard with missing or misplaced joints. Enriching our dataset with more high-quality rigged models is expected to improve the overall performance. (2) Our method currently works for the humanoid mesh, and it will be interesting to extend our approach to the quadruped.

### CRedit authorship contribution statement

**Jing Ma:** Investigation, Methodology, Software, Writing – original draft. **Dongliang Zhang:** Funding acquisition, Project administration, Supervision, Writing – review & editing.

### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

### Data availability

Data will be made available on request.

### Acknowledgments

This work is supported in part by the National Natural Science Foundation under Grant 61972340.

**Table 8**  
Time comparisons of RigNet and Ours.

| #V    | Running time/s |       |
|-------|----------------|-------|
|       | RigNet         | Ours  |
| 1000  | 84.96          | 19.16 |
| 2000  | 89.80          | 21.01 |
| 3000  | 120.94         | 22.49 |
| 4000  | 134.39         | 24.03 |
| 5000  | 175.44         | 25.98 |
| 6000  | 224.74         | 27.94 |
| 7000  | 333.66         | 30.38 |
| 8000  | 461.31         | 33.33 |
| 9000  | 598.59         | 34.31 |
| 10000 | 715.25         | 36.90 |
| 11000 | 872.38         | 42.28 |
| 12000 | 1654.68        | 45.42 |

## Appendix A. Appendices

As shown in Table 7, the GNN backbone stacks three *GMEdgeConv* followed by a *MLP-MaxPool*, in which *GMEdgeConv* extract local features around the vertex and the *MLP-MaxPool* extract the global features of the mesh. We use the combination of local features and global features since it is demonstrated to yield better performance [7].

## Appendix B. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.cag.2023.05.018>.

## References

- [1] Poirier M, Paquette E. Rig retargeting for 3D animation. In: Graphics interface. 2009, p. 103–10.
- [2] Pantuwong N, Sugimoto M. A novel template-based automatic rigging algorithm for articulated-character animation. *Comput Animat Virtual Worlds* 2012;23(2):125–41.
- [3] Baran I, Popović J. Automatic rigging and animation of 3d characters. *ACM Trans Graph* 2007;26(3):72–es.
- [4] Pascucci V, Scorzelli G, Bremer P-T, Mascarenhas A. Robust on-line computation of Reeb graphs: Simplicity and speed. In: *ACM SIGGRAPH 2007 papers*. 2007, p. 58–es.
- [5] Pantuwong N, Sugimoto M. Skeleton-growing: A vector-field-based 3D curve-skeleton extraction algorithm. In: *ACM SIGGRAPH ASIA 2010 sketches*. 2010, p. 1–2.
- [6] Xu Z, Zhou Y, Kalogerakis E, Singh K. Predicting animation skeletons for 3d articulated models via volumetric nets. In: *2019 international conference on 3D vision*. IEEE; 2019, p. 298–307.
- [7] Xu Z, Zhou Y, Kalogerakis E, Landreth C, Singh K. Rignet: Neural rigging for articulated characters. 2020, arXiv preprint arXiv:2005.00559.
- [8] Xu Z, Zhou Y, Yi L, Kalogerakis E. Morig: Motion-aware rigging of character meshes from point clouds. In: *SIGGRAPH Asia 2022 conference papers*. 2022, p. 1–9.
- [9] Dionne O, de Lasa M. Geodesic voxel binding for production character meshes. In: *Proceedings of the 12th ACM SIGGRAPH/eurographics symposium on computer animation*. 2013, p. 173–80.
- [10] Liu L, Zheng Y, Tang D, Yuan Y, Fan C, Zhou K. Neuroskin: Automatic skin binding for production characters with deep graph networks. *ACM Trans Graph* 2019;38(4):1–12.
- [11] Autodesk. Quick rig tool. 2022, URL <https://knowledge.autodesk.com/support/maya/learn-explore/caas/CloudHelp/cloudhelp/2022/ENU/Maya-CharacterAnimation/files/GUID-DC29C982-D04F-4C20-9DBA-4BBB33E027EF-htm.html>.
- [12] Adobe. Upload and rig 3D characters with Mixamo. 2022, URL <https://helpx.adobe.com/creative-cloud/help/mixamo-rigging-animation.html>.
- [13] Kavan L, Collins S, Žára J, O'Sullivan C. Skinning with dual quaternions. In: *Proceedings of the 2007 symposium on interactive 3D graphics and games*. 2007, p. 39–46.
- [14] Kavan L, Collins S, Žára J, O'Sullivan C. Geometric skinning with approximate dual quaternion blending. *ACM Trans Graph* 2008;27(4):1–23.

- [15] Jacobson A, Baran I, Popovic J, Sorkine O. Bounded biharmonic weights for real-time deformation. *ACM Trans Graph* 2011;30(4):78.
- [16] Kavan L, Sorkine O. Elasticity-inspired deformers for character articulation. *ACM Trans Graph* 2012;31(6):1–8.
- [17] Dionne O, De Lasa M. Geodesic binding for degenerate character geometry using sparse voxelization. *IEEE Trans Vis Comput Graphics* 2014;20(10):1367–78.
- [18] Wang Y, Jacobson A, Barbič J, Kavan L. Linear subspace design for real-time shape deformation. *ACM Trans Graph* 2015;34(4):1–11.
- [19] Le BH, Lewis J. Direct delta mush skinning and variants. *ACM Trans Graph* 2019;38(4):113.
- [20] Pan X, Huang J, Mai J, Wang H, Li H, Su T, et al. HeterSkinNet: A heterogeneous network for skin weights prediction. In: *Proceedings of the ACM on computer graphics and interactive techniques*, Vol. 4, no. 1. Association for Computing Machinery; 2021.
- [21] Bronstein MM, Bruna J, LeCun Y, Szlam A, Vandergheynst P. Geometric deep learning: going beyond euclidean data. *IEEE Signal Process Mag* 2017;34(4):18–42.
- [22] Wu Z, Pan S, Chen F, Long G, Zhang C, Philip SY. A comprehensive survey on graph neural networks. *IEEE Trans Neural Netw Learn Syst* 2020;32(1):4–24.
- [23] Zhang Z, Cui P, Zhu W. Deep learning on graphs: A survey. *IEEE Trans Knowl Data Eng* 2020.
- [24] Zhou J, Cui G, Hu S, Zhang Z, Yang C, Liu Z, et al. Graph neural networks: A review of methods and applications. *AI Open* 2020;1:57–81.
- [25] Boscaini D, Masci J, Rodolà E, Bronstein M. Learning shape correspondence with anisotropic convolutional neural networks. *Adv Neural Inf Process Syst* 2016;29.
- [26] Monti F, Boscaini D, Masci J, Rodola E, Svoboda J, Bronstein MM. Geometric deep learning on graphs and manifolds using mixture model cnns. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, p. 5115–24.
- [27] Yi L, Su H, Guo X, Guibas LJ. Syncspecnn: Synchronized spectral cnn for 3d shape segmentation. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, p. 2282–90.
- [28] Hanocka R, Hertz A, Fish N, Giryas R, Fleishman S, Cohen-Or D. Meshcnn: A network with an edge. *ACM Trans Graph* 2019;38(4):1–12.
- [29] Masci J, Boscaini D, Bronstein M, Vandergheynst P. Geodesic convolutional neural networks on riemannian manifolds. In: *Proceedings of the IEEE international conference on computer vision workshops*. 2015, p. 37–45.
- [30] Verma N, Boyer E, Verbeek J. Feastnet: Feature-steered graph convolutions for 3d shape analysis. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, p. 2598–606.
- [31] Wang Y, Sun Y, Liu Z, Sarma SE, Bronstein MM, Solomon JM. Dynamic graph cnn for learning on point clouds. *Acm Trans Grap (Tog)* 2019;38(5):1–12.
- [32] Sudre CH, Li W, Vercauteren T, Ourselin S, Jorge Cardoso M. Generalised dice overlap as a deep learning loss function for highly unbalanced segmentations. In: *Deep learning in medical image analysis and multimodal learning for clinical decision support*. Springer; 2017, p. 240–8.
- [33] Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, et al. Pytorch: An imperative style, high-performance deep learning library. *Adv Neural Inf Process Syst* 2019;32.
- [34] Fey M, Lenssen JE. Fast graph representation learning with PyTorch geometric. 2019, arXiv preprint [arXiv:1903.02428](https://arxiv.org/abs/1903.02428).
- [35] Thekumparampil KK, Wang C, Oh S, Li L-J. Attention-based graph neural network for semi-supervised learning. 2018, arXiv preprint [arXiv:1803.03735](https://arxiv.org/abs/1803.03735).